

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №7 по курсу «Дискретный анализ»

Студент: Д. О. Кочев
Преподаватель: А. А. Кухтичев
Группа: М8О-306Б-22
Дата:
Оценка:
Подпись:

Москва, 2024

Лабораторная работа №7

Задача: Задан прямоугольник с высотой n и шириной m , состоящий из нулей и единиц. Найдите в нём прямоугольник наибольшей площади, состоящий из одних нулей.

Формат ввода

В первой строке заданы $1 \leq n \leq 500$ и $1 \leq m \leq 500$. В последующих n строках записаны по m символов 0 или 1 - элементы прямоугольника.

Формат вывода

Необходимо вывести одно число – максимальную площадь прямоугольника из одних нулей.

1 Описание

Здесь требуется применить динамическое программирование для задачи максимизации. Для решения сначала пройдемся по введенной матрице, и для каждой нулевой ячейки определим, сколько 0-ей находятся над ней. Для этого нам понадобится $\Theta(n * m)$ операций, где n - количество строк, а m - количество столбцов.

В итоге мы получим матрицу, в которой каждая строка - основание гистограммы. Затем нам останется пройти по каждой строке, и подсчитать площадь самого большого прямоугольника в гистограмме, а потом выбрать максимальную.

Для решения задачи с площадью будем использовать стек. Если следующий столбец гистограммы больше, чем предыдущий, то добавляем его в стек. Если меньше или равен, то будем удалять из стека до тех пор, пока верхушка стека не станет меньше. Параллельно будем считать площадь перед каждым удалением по формуле $(i - x) * h_x$, где i - индекс столбца, который оказался меньше предыдущего, а x и h_x - индекс и высота столбца, который мы собираемся удалить из стека. Выбрав максимальную площадь из всех полученных мы узнаем ответ на задачу.

Общая сложность такого решения будет равна $\Theta(n * m)$.

2 Исходный код

Мой код получился небольшим и поместился в одну функцию *main*. Превращение матрицы в массив гистограмм я реализую с помощью динамического программирования.

```
1   for (int i = 0; i < n; i++){
2       for (int j = 0; j < m; j++){
3           if (arr[i][j] == 0){
4               if (i == 0){
5                   arr[i][j] = 1;
6               }
7               else{
8                   arr[i][j] = 1 + arr[i - 1][j];
9               }
10          }
11          else{
12              arr[i][j] = 0;
13          }
14      }
15  }
16 }
```

Затем я реализовываю алгоритм, который ищет максимальный прямоугольник в каждой гистограмме.

```
1   int answer = 0;
2   int max_line = 0;
3   std::stack<std::pair<int, int>> gistogram;
4   gistogram.push({-1, -1}); // i, h
5   for (int i = 0; i < n; i++){
6       for (int j = 0; j < m + 1; j++){
7           std::pair<int, int> elem = gistogram.top();
8           if (j == m){
9               gistogram.push({j, 0});
10          }
11          if (arr[i][j] > elem.second && j != m){
12              gistogram.push({j, arr[i][j]});
13          }
14          else{
15              int j_elem = arr[i][j];
16              if (j == m){
17                  j_elem = 0;
18              }
19              int test_max = 0;
20              while (j_elem <= gistogram.top().second){
21                  elem = gistogram.top();
```

```

22         test_max = (j - elem.first) * elem.second;
23         if (test_max > max_line){
24             max_line = test_max;
25         }
26         gistogram.pop();
27     }
28     gistogram.push({elem.first, arr[i][j]});
29     if (answer < max_line){
30         answer = max_line;
31     }
32     max_line = 0;
33 }
34 }
35 gistogram.pop();
36 }

```

Ответ после этого цикла будет храниться в переменной *answer*.

3 Консоль

```
dmitrykochev$ cat test1.txt
4 5
01011
10001
01000
11011
dmitrykochev$ ./laba7 <test1.txt
4
dmitrykochev$ cat test2.txt
5 6
000000
000000
000000
000000
000000
dmitrykochev$ ./laba7 <test2.txt
30
dmitrykochev$ cat test3.txt
5 6
100100
100011
000111
011111
100100
dmitrykochev$ ./laba7 <test2.txt
6
```

4 Тест производительности

Тест производительности представляет из себя следующее: одни и те же тесты мы будем использовать как в моем решении, так и в наивном алгоритме. В наивном алгоритме мы будем для каждого столбца идти максимально влево и максимумально вправо, пока не встретим столбец, ниже нашего. Если сделать так для каждого столбца и сохранить максимальную площадь из всех полученных, мы найдем площадь максимального прямоугольника в гистограмме. Это решение займет $\Theta(n * m + n * m^2)$. Тестов будет три, тест характеризуется двумя числами n и m , а также матрицей размеров $m * n$. В первый тесте $n = 100$ и $m = 100$. Во втором $n = 250$ и $m = 250$. В третьем тесте $n = 500$ и $m = 500$.

```
dmitrykochev$ ./benchmark <test1.txt
Stack algorithm time: 8335us
Native algorithm time: 8589us
dmitrykochev$ ./benchmark <test2.txt
Stack algorithm time: 40356us
Native algorithm time: 51771us
dmitrykochev$ ./benchmark <test3.txt
Stack algorithm time: 117640us
Native algorithm time: 238876us
```

Как можно заметить, при увеличении размеров матрицы алгоритм с использованием стека становится заметно быстрее наивного алгоритма.

Действительно, как говорилось выше, мой алгоритм имеет сложность $\Theta(n * m)$, а наивный $\Theta(n * m + n * m^2)$. Это подтверждает тест производительности.

5 Выводы

Выполнив седьмую лабораторную работу по курсу «Дискретный анализ», я научился применять динамическое программирование на практике. Это было не очень просто. Метод требует фокусировки на задаче и нестандартного мышления, можно сказать, олимпиадного. Но мне удалось правильно применить данный метод, а так же решить сопутствующую непростую задачу. Убежден, что новые знания и опыт пригодятся мне в будущем.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))